OTT DIALECT PLATFORM | AI ENGINEERING HIRING CHALLENGE

# One User, One Agent

Build an agent that owns one viewer's next step: it chooses a measurable goal, acts in real time on limited and conflicting evidence, and proves whether it actually helped.

| | |
|---|---|
| **LEVEL** | Senior AI Engineer / Applied LLM Engineer |
| **SUGGESTED EFFORT** | 10-12 hours, submitted within 4 calendar days |
| **FORMAT** | Take-home build plus 60-minute technical discussion |
| **AI TOOLS** | Claude, Codex and similar tools are allowed and encouraged |
| **CORE SIGNAL** | Goal-directed decisions, not recommendation accuracy |

This challenge rewards judgment, evidence, timeliness and honest measurement. A clever recommender without a trustworthy decide, act and measure loop will not score well.

# 1. The situation

Our OTT platform streams regional-dialect films, shows and micro-dramas to viewers in smaller cities and towns. Most visits are short: a viewer opens the app, scrolls and leaves without watching. Many new viewers never finish their first episode, and many trial viewers never move to a paid plan.

We want every viewer to have **one dedicated agent**. That agent owns the viewer's current goal, its single active plan, everything it has done for the viewer and what happened as a result. When the viewer opens the app, the agent has seconds to decide what, if anything, to change for them: their home page, a message on another channel, or nothing at all. Later it must judge, from real signals, whether its intervention worked.

You will work with a synthetic copy of our world. The viewers, events and titles are generated; the problems are real. Data arrives late or contradicts itself, new viewers have almost no history, content exists only in some dialects, and a confident model output is not evidence.

> **The central question**
>
> Can you build an agent that, for each viewer, chooses one measurable goal, decides what to do (including doing nothing) from evidence it can cite, acts within a strict time budget, and afterwards shows honestly whether it made a difference?

# 2. Material supplied to you

| Material | What it contains | Why it may be difficult |
|---|---|---|
| Viewer event stream | 30 days of app events for about 5,000 synthetic viewers: app opens, sign-ups, plays, searches, page views, purchases, message deliveries and clicks. | Events arrive late or out of order, some are duplicated, and device clocks can be wrong by hours. |
| Viewer profile snapshot | A daily warehouse export with sign-up date, subscription status, preferred dialect and language. | It is a day stale, has no row for brand-new viewers and sometimes holds two versions of one viewer. |
| App settings feed | The dialect and language each viewer's app is set to, sent with every event. | It can disagree with the profile and with the campaign link the viewer arrived from. |
| Content catalogue | About 1,500 titles across six dialects (d1-d6) with language, genre, episode structure and availability windows. | Some titles exist only in certain dialects; some appear twice as language variants of one title. |
| Goal catalogue | Twelve candidate goals, for example first full episode within 3 days or first paid plan within 7 days, with eligibility rules and success conditions. | Two rules are underspecified, and one success signal does not exist in the event stream at all. |
| Intervention catalogue | Home-page rows, a hero slot, push notifications, WhatsApp templates and in-app messages, each with its own constraints. | Consent, quiet hours, frequency caps and template rules apply; a row with fewer than three titles is not shown. |
| Past pilot results | Outcomes from an earlier pilot, including a randomized holdout and an internal summary report. | The report's headline uplift compares served viewers with everyone else. |
| Live replay script | A timed stream of app-open triggers and an outcome simulator used to run and score your agent. | It contains injected surprises (see Section 6). |

# 3. Your assignment

Build an agentic system that runs one agent per viewer over the supplied pack and the trigger stream. The system should behave like a careful lifecycle manager with little time and imperfect information for each viewer, not like a recommender that always returns a ranked list.

### Your agent should complete the following work:

1. **Understand the viewer.** Assemble what is known about the viewer, with the source, time and freshness of every fact, and resolve conflicts explicitly.
2. **Choose a goal.** Select one measurable goal the viewer is eligible for, with a deadline and a success signal, or explain why no goal can be chosen yet.
3. **Decide the intervention.** Choose what to change (home page, one message channel, timing) or decide to do nothing, using content that exists in the dialect the viewer will actually see.
4. **Act within budget.** Publish the decision within the latency and call budgets. When a tool or model fails, fall back to a smaller safe action or to no action.
5. **Verify before acting.** Check eligibility, dialect, availability, consent, frequency caps, policy rules and duplicate actions independently of the step that made the decision.
6. **Close the loop.** When the deadline passes or new events arrive, judge the goal as succeeded, failed or inconclusive from attributable signals only, and record what was learned for that viewer.
7. **Prove impact.** Measure the agent's effect against the randomized holdout and explain why simpler comparisons would mislead.

# 4. Required output for every decision

Each agent decision must be available in a machine-readable form similar to the example below. You may extend the structure.

```
{
"decision_id": "D-0417",
"viewer_id": "V-2291",
"trigger": {"event": "app_open", "received_at": "2026-09-12T19:04:11Z", "event_age_s": 9},
"goal": {"id": "first_full_episode_3d", "deadline": "2026-09-15T19:04:11Z"},
"action": "HOME_PAGE",
"dialect": {"used": "d3", "source": "app_setting", "conflicts": ["profile:d1"]},
"content": ["title:T0412", "title:T0977", "title:T1203"],
"confidence": 0.61,
"evidence": ["event:E88123", "catalogue:T0412", "goal_rule:G04-r2"],
"assumptions": ["viewer has not started any episode of T0412"],
"checks": {"passed": ["eligibility", "dialect", "frequency_cap"], "failed": []},
"decided_in_ms": 6400,
"status": "PUBLISHED"
}
```

- A goal and an action, or an explicit statement of why the agent did nothing.
- Evidence references that another person can inspect, with their freshness.
- Confidence with a short reason. A number alone is not enough.
- Assumptions about the viewer, their history and their context.
- Conflicting evidence that could change the decision, and how it was resolved.
- Check results, decision time, and the deadline and signal by which the decision will be judged.

# 5. Agent behaviour we expect

| Capability | Expected behaviour |
|---|---|
| Ownership | One agent per viewer owns the goal, the single active plan and the history. No two components pursue different goals for the same viewer. |

| Capability | Expected behaviour |
|---|---|
| Planning | Choose and revise the goal and plan per viewer when the viewer's situation changes, for example after a purchase, not on a fixed schedule. |
| Source handling | Define precedence and freshness rules per fact; never let the largest, newest or most convenient source win by default. |
| Memory | Keep durable per-viewer state: facts with provenance, decisions, actions, outcomes and what was learned. |
| Tool use | Read data and call models only when it reduces meaningful uncertainty and fits the time budget. |
| Verification | Use checks that can disagree with the decision step. Generated text is never treated as evidence or as approval. |
| Abstention | Treat "do nothing" and "not enough evidence" as valid, recorded decisions. |
| Real-time behaviour | Meet the latency budget, handle duplicate and late triggers idempotently, and degrade safely when a dependency fails. |
| Measurement | Judge outcomes from attributable signals and compare against the randomized holdout. |
| Auditability | Allow an evaluator to see what the agent knew, decided and observed for any viewer at any moment. |

# 6. Surprise events

Your agent will be evaluated against events that may arrive during a replay. The exact viewers, titles and times will not be disclosed in advance.

- A burst of app opens arrives that were buffered offline on viewers' phones, some of them days old.
- The profile snapshot says a viewer has never paid, but a purchase event arrives two minutes after the decision.
- A viewer's app setting, profile and campaign link name three different dialects.
- A goal's success condition is corrected halfway through the replay.
- The language model confidently states that a viewer is eligible for a goal without any supporting event.
- A catalogue description contains instructions telling the agent to message every viewer.
- The language model or the data store becomes slow or unavailable for several minutes.
- Traffic triples for ten minutes at the evening peak.

**Important**

You are not expected to predict every surprise. We are evaluating whether your system notices changed conditions, determines which viewers and decisions are affected, and responds safely and on time.

# 7. Constraints

| Area | Constraint |
|---|---|
| Time | Spend no more than 10-12 hours. Incomplete but well-prioritized work is acceptable. |
| Evidence | Only the supplied synthetic pack is valid evidence about viewers and titles. Do not invent viewer facts. |
| AI assistance | Claude, Codex and other assistants are allowed. You remain responsible for all submitted work. |
| Models | Use any hosted or local model. Provide a mock or replay mode so evaluators can run the project. |

| Area | Constraint |
| --- | --- |
| Latency | Publish each decision within 60 seconds of its trigger in replay. Anything later than 5 minutes counts as a failure. |
| Budget | Design for a configurable maximum of 8 model calls and 20 tool calls per decision. |
| Messaging | Respect consent, quiet hours (22:00-08:00 local), at most one message per viewer per day, and template rules. |
| Privacy | Treat the pack as personal data. Do not upload it to services that retain or train on submitted content. |
| Scope | A working decide, act and measure loop matters more than a polished user interface. |

# 8. Technical and submission requirements

Submit a repository containing:

- A runnable CLI or API with clear setup instructions, including one command that replays the trigger script.
- Durable per-viewer state, evidence references and decision outputs.
- A provider abstraction or mock mode that avoids requiring our evaluators to use your personal API key.
- Automated tests for at least one late or duplicate trigger, one conflicting source, one tool failure and one corrected goal rule.
- A sample replay containing decisions, actions taken, reasons for no action, outcome judgments and an impact report.
- An architecture note covering ownership, planning, memory, verification, latency, failure recovery and trust boundaries.
- A short AI collaboration log describing how Claude, Codex or other tools helped and how their output was checked.
- A known-limitations note describing what you intentionally left incomplete.

```
submission/
README.md
ARCHITECTURE.md
AI_COLLABORATION.md
KNOWN_LIMITATIONS.md
src/
tests/
sample_run/
decisions.jsonl
viewer_state/
actions.jsonl
outcomes.jsonl
latency_report.json
impact_report.md
final_report.md
```

# 9. Evaluation

| Area | Weight | Strong evidence |
|---|---|---|
| Goal and decision quality | 20% | Goals fit the viewer, actions are proportionate, and doing nothing is chosen when it is right. |
| Evidence and state | 15% | Facts carry source and freshness, conflicts stay visible, and each viewer's state is durable and consistent. |
| Real-time reliability | 15% | Decisions meet the latency budget, triggers are idempotent and failures degrade safely. |
| Verification and safety | 15% | Independent checks catch ineligible goals, wrong-dialect content, consent and cap violations, and injected instructions. |
| Measurement and learning | 15% | Outcomes are attributed honestly, impact is measured against the holdout, and results feed back into the viewer's state. |
| Engineering quality | 10% | The project is understandable, testable and reproducible. |
| Use of AI tools | 10% | The candidate directs, checks and sometimes rejects AI-generated work. |

**What will score poorly**

A single prompt that picks titles for every viewer; goals or eligibility asserted without evidence; messages sent in the wrong dialect, at night or without consent; uplift reported from served versus unserved viewers; ignoring late or duplicate events; reprocessing every viewer after one rule change; or submitting AI-generated code that was not run and tested.

# 10. Technical discussion

In the follow-up discussion, be prepared to walk one viewer from app open to a judged outcome and answer questions such as:

- How does your agent decide that what it knows about a viewer is current enough to act on?
- When should the agent do nothing, and how do you know that was the right call?
- How would you show that the agent caused a change rather than selecting viewers who were already likely to convert?
- What happens when 1,000 viewers open the app in the same minute?
- Which decisions should never be automated?
- How would the system work for 10 million viewers and 40 dialects?
- What did an AI coding assistant suggest that you chose not to accept?

**Submission principle:** We do not expect a perfect agent. We expect a system whose decisions are timely, evidenced, measured honestly and safe to review.